

MPCS 57200 1 Generative AI

Spring 2025

HW for Lec1

Yiming Cheng

Cnet-id:12450588

MPCS 57200 1 Generative AI

Spring 2025

HW for Lec1

File Structure

HW1-1

`gpt2_example.ipynb`

`hello_gpt4o.ipynb`

HW1-2

HW1-3

File Structure

Lec1_HW_YimingCheng_12450588

```
|— HW1-1
|   |— gpt2_example.ipynb
|   |— gpt2_example.pdf
|   |— hello_gpt4o.ipynb
|   |— hello_gpt4o.ipynb - Colab.pdf
|— HW1-3
|   |— 2096.MP4
|— readme.md
|— readme.pdf
```

HW1-1

Get the gpt2_example.ipynb and hello_gpt4o.ipynb Jupyter notebooks running

- Could be on your computer or a service like the excellent free google colab or the CS computers

Submit evidence that you have done so

- The Print Preview command in Jupyter could be helpful
- Other evidence like screenshots are ok too

Note: Do not put any API keys in your source code. Either use environment variables (see python-dotenv) or a secrets manager like the one I demonstrated for Google colab in class

For extra credit: Explore and extend (e.g., different queries, different roles, etc.) and report on your findings

gpt2_example.ipynb

Environment: The environment files is in `HW1-1/.venv`

Output: The answer to the original transformer prediction is

The answer to the universe is not to be found in the **universe**

```
print(predicted_text)
```

✓ 0.0s

```
The answer to the universe is not to be found in the universe
```

Extra:

1. On Apple Silicon (mine is M3 pro), PyTorch supports Metal Performance Shaders (MPS) as a device for accelerated computation. I use mps instead of cuda:

```
# tokens_tensor = tokens_tensor.to('cuda')
# model.to('cuda')

tokens_tensor = tokens_tensor.to('mps')
model.to('mps')
```

✓ 0.2s

2. To explore the probability distribution provided by the transformer model and examine tokens other than “universe,” I modify the code to print the top-ranked tokens and their probabilities:

```
# Extract probabilities for the next token
next_token_logits = predictions[0, -1, :]
probabilities = torch.softmax(next_token_logits, dim=-1)

# Decode all tokens and probabilities
top_k = 10 # Number of top tokens to display
sorted_indices = torch.argsort(probabilities, descending=True)

# Print the top tokens and their probabilities
print("Top tokens excluding 'universe':")
for idx in sorted_indices[:top_k]:
    token = tokenizer.decode([idx.item()]).strip()
    if token != "universe": # Exclude the token "universe"
        print(f"Token: '{token}', Probability: {probabilities[idx]:.4f}")
```

✓ 0.9s

and the result:

```
Top tokens excluding 'universe':  
Token: 'Bible', Probability: 0.0311  
Token: 'laws', Probability: 0.0157  
Token: 'physical', Probability: 0.0127  
Token: '"', Probability: 0.0103  
Token: 'cosmos', Probability: 0.0082  
Token: 'form', Probability: 0.0079  
Token: 'first', Probability: 0.0074  
Token: 'very', Probability: 0.0065  
Token: 'simple', Probability: 0.0060
```

interesting

3. To generate completions with varying lengths (e.g., 1 token, 5 tokens, 10 tokens, 20 tokens), I modify the code to generate tokens chains:

```

# Generate text of varying lengths
device = torch.device("mps")
# Function to generate text with a specified number of tokens
def generate_text(input_text, num_tokens, model, tokenizer, device):
    indexed_tokens = tokenizer.encode(input_text)
    tokens_tensor = torch.tensor([indexed_tokens]).to(device)

    model.eval()
    with torch.no_grad():
        for _ in range(num_tokens):
            outputs = model(tokens_tensor)
            predictions = outputs[0]
            predicted_index = torch.argmax(predictions[0, -1, :]).item()
            indexed_tokens.append(predicted_index)
            tokens_tensor = torch.tensor([indexed_tokens]).to(device)

    return tokenizer.decode(indexed_tokens)

# Input text
input_text = "The answer to the universe is not to be found in the"

# Generate texts of different lengths
lengths = [1, 5, 10, 20]
for length in lengths:
    generated_text = generate_text(input_text, length, model, tokenizer, device)
    print(f"\nGenerated text ({length} tokens):\n{generated_text}")

```

✓ 5.2s

Result:

```

Generated text (1 tokens):
The answer to the universe is not to be found in the universe

Generated text (5 tokens):
The answer to the universe is not to be found in the universe itself, but in

Generated text (10 tokens):
The answer to the universe is not to be found in the universe itself, but in the universe itself.

Generated text (20 tokens):
The answer to the universe is not to be found in the universe itself, but in the universe itself.

The universe is not a place of infinite numbers

```

4. various queries and roles:

```

# Generate text for a single role and query
role = "scientist"
query = "explain the concept of time."

# Combine role and query
input_text = f"As a {role}, {query}"
indexed_tokens = tokenizer.encode(input_text)
tokens_tensor = torch.tensor([indexed_tokens]).to(device)

# Generate text
num_tokens = 20
with torch.no_grad():
    for _ in range(num_tokens):
        outputs = model(tokens_tensor)
        predictions = outputs[0]
        predicted_index = torch.argmax(predictions[0, -1, :]).item()
        indexed_tokens.append(predicted_index)
        tokens_tensor = torch.tensor([indexed_tokens]).to(device)

# Decode and print
generated_text = tokenizer.decode(indexed_tokens)
print(f"Role: {role}, Query: {query}\nGenerated Text:\n{generated_text}")

```

✓ 2.6s

results:

```

Role: scientist, Query: explain the concept of time.
Generated Text:
  As a scientist, explain the concept of time.

The concept of time is a concept that is often used to describe the time of a person

```

all result can be found in [HW1-1/gpt2_example.pdf](#)

hello_gpt4o.ipynb

Environment: The environment files is in [google colab](#)

Output:

```
[8] print(completion)

ChatCompletion(id='chatcmpl-Ap5FDokqhWgfaShrBQDpf0VKI8pC7', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='Certainly! The story of the frog and the scorpion is a classic fable that illustrates the idea that some creatures cannot change their inherent nature.\n\nIn the story, a scorpion wants to cross a river but cannot swim. It asks a frog to carry it across on its back. The frog is hesitant, fearing that the scorpion might sting it during the journey. The scorpion reassures the frog, arguing that if it were to sting the frog, they would both drown. Convinced by this logic, the frog agrees to help.\n\nMidway across the river, the scorpion stings the frog. As they both begin to sink, the frog asks the scorpion why it stung him, knowing the consequences. The scorpion replies, "I couldn\'t help it. It\'s in my nature."\n\nThe fable highlights themes of trust, betrayal, and the idea that some behaviors are innate and unchangeable.', refusal=None, role='assistant', audio=None, function_call=None, tool_calls=None))], created=1736738251, model='gpt-4o-2024-08-06', object='chat.completion', service_tier='default', system_fingerprint='fp_703d4ff298', usage=CompletionUsage(completion_tokens=187, prompt_tokens=28, total_tokens=215, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0, audio_tokens=0, reasoning_tokens=0, rejected_prediction_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
```

```
[9] print(completion.choices[0].message.content)

Certainly! The story of the frog and the scorpion is a classic fable that illustrates the idea that some creatures cannot change their inherent nature.

In the story, a scorpion wants to cross a river but cannot swim. It asks a frog to carry it across on its back. The frog is hesitant, fearing that the scorpion might sting it during the journey. The scorpion reassures the frog, arguing that if it were to sting the frog, they would both drown. Convinced by this logic, the frog agrees to help.

Midway across the river, the scorpion stings the frog. As they both begin to sink, the frog asks the scorpion why it stung him, knowing the consequences. The scorpion replies, "I couldn't help it. It's in my nature."

The fable highlights themes of trust, betrayal, and the idea that some behaviors are innate and unchangeable.
```

Extra:

I create a unique story by assigning it the role of a “wildly imaginative storyteller blending mythology with science fiction.” The prompt asked for a tale where a modern scientist meets Zeus. I set the temperature to 0.8 to encourage creative and diverse responses, exploring AI’s storytelling potential.

```
hello_gpt4o.ipynb ☆
文件 修改 视图 插入 代码执行程序 工具 帮助
+ 代码 + 文本

rolecompletion = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.8,
    messages=[
        {"role": "system", "content": "You are a wildly imaginative storyteller who blends mythology with science fiction."},
        {"role": "user", "content": "Create a new story where a modern scientist encounters Zeus."}
    ]
)

# Print the generated story from the completion
print(rolecompletion.choices[0].message.content)

In the heart of New York City, where skyscrapers clawed at the belly of the heavens, Dr. Amelia Grant, a renowned astrophysicist, was hard at work, probing the mysteries of the universe. Her latest obsession was the study of cosmic thunder—a rare and enigmatic phenomenon that defied the known laws of physics. Its origins were a mystery, and its implications, potentially groundbreaking.

One evening, as Amelia analyzed data from the latest thunderstorm captured by her array of satellite sensors, she noticed an anomaly: a surge of energy unlike any she had ever seen. It was as if the sky itself had momentarily cracked open, releasing a torrent of raw, celestial power. Intrigued and puzzled, Amelia decided to investigate further.

Her research led her to the remote peaks of Mount Olympus, not the legendary one from Greek myth, but its geographical cousin in Greece. She hypothesized that the anomaly might be connected to the mountain's unique electromagnetic properties, unnoticed by modern science until now.

Upon arriving at the foot of the mountain, Amelia felt a strange, electric pulse in the air. The climb was arduous, but the allure of unraveling a cosmic mystery drove her onward. As she reached the summit, a sudden storm gathered, dark clouds swirling above with unnatural speed.

Amelia stood her ground as thunder rumbled ominously, the air crackling with energy. As she raised her scientific instruments to measure the atmospheric conditions, a blinding flash of lightning struck the peak. Disoriented, she looked up to find herself face to face with a figure of immense stature—a man draped in flowing robes, his eyes blazing like twin suns amidst a sky of tumultuous clouds.

"Who dares to disturb the realm of the gods?" boomed the figure, his voice echoing through the mountains.

Amelia, initially dumbfounded, quickly regained her composure. "I am Dr. Amelia Grant, a scientist from Earth," she stammered, "I seek the source of the cosmic thunder."

The figure's laughter roared like a tempest. "You seek the power of Zeus, king of the gods," he declared, gesturing to the heavens. "It seems your world has forgotten what was once known by all."

Zeus, the revelation hit Amelia like a bolt of electrifying clarity. The ancient god of thunder himself. The cosmic thunder wasn't a natural phenomenon—it was the remnants of divine energy, bleeding through dimensions as the old gods grew restless in their mythic slumber.

"Why now?" Amelia asked, her scientific curiosity mingling with newfound awe. "Why reveal yourself in this era?"

Zeus's expression softened, a flicker of ancient wisdom in his gaze. "The world is changing," he said. "The boundaries between realms grow thin, and the gods stir once more. Your science seeks answers, and it is time we remind mankind of the magical truths that once shaped your world."

Amelia realized she stood at the cusp of an unprecedented discovery—an alliance between science and myth, a bridge between the empirical and the ethereal. Zeus, sensing her understanding, offered a pact: to share the knowledge of the cosmic thunder, allowing humanity to harness its potential responsibly in return for rekindling the stories of old, ensuring the gods were remembered in the modern age.

With their bond forged beneath the stormy sky, Amelia returned to New York, her research taking a revolutionary turn. Papers were published, marvels were unveiled, and the tale of her encounter with Zeus became a legend in its own right—a testament to the limitless possibilities when science embraces the mythic tapestry of the universe.

And somewhere high above, in the realm of Olympus reborn, Zeus watched with a satisfied smile, knowing that the world of mortals had not been forgotten by the gods.
```

the complete result is in `HW1-1/hello_gpt4o.ipynb - Colab.pdf`

HW1-2

You are looking at a large number of documents that are almost all in English, and want to identify which ones are not

For each of the following, say whether it is statistics, machine learning, deep learning, AI, Generative AI, or AGI (**Note:** you are not expected to implement, just say which it is)

- Ask ChatGPT which documents are in English
- Zip the collection of documents and flag the ones that don't compress well
 - zipping algorithms are designed to code the most common text efficiently, so it can be useful for anomaly detection
 - We will learn the math behind this next week because it turns out to be central to Gen AI
- Use a random forest to identify the anomalies
- Train a deep learning model for anomaly detection based on a training set of english and non-english documents

1. Generative AI

ChatGPT uses the Transformer architecture to predict the probability distribution of the next token based on existing tokens (in this case, documents in different languages). It is a type of Generative Artificial Intelligence (Generative AI), making it a form of Generative AI.

2. Statistics

Documents in different languages exhibit certain probability distributions or statistical patterns based on compression rates. Encoding common patterns while identifying and compressing files that compress badly, which are more likely to be non-English documents, is a statistical method.

3. Machine Learning

Random forests are used to analyze document compression rates. By training on labeled datasets of English and non-English documents, they learn the characteristic patterns of English documents. Then, through the integration of multiple decision trees, they classify unlabeled documents and identify anomalies that significantly differ from English characteristics. Therefore, it is a machine learning method.

4. Deep Learning

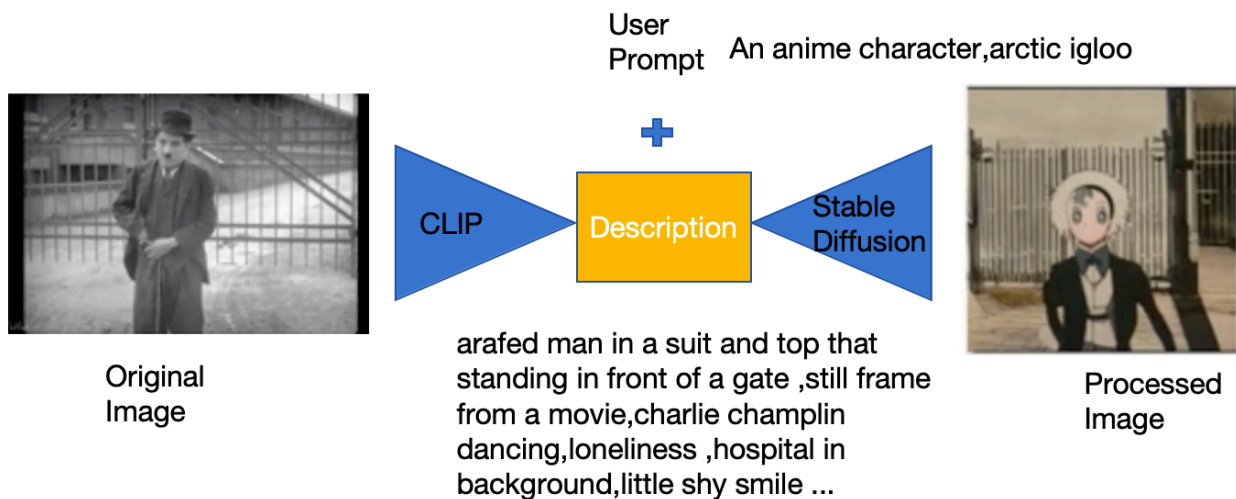
It identifies unusual documents by learning the patterns of English and non-English documents during training. Its ability to find complex features and work with large amounts of data makes it a good choice for this type of anomaly detection, so it is considered deep learning.

HW1-3

Try out a Generative AI based tool, investigate it, and report on your results.

- A couple of paragraphs or a few bullets is OK as long as they are insightful
- Be prepared to talk for 2 minutes in class on what you did and discovered

While it could be ChatGPT, try to be more creative along the lines of what I did with DropDeck and the Text-to-3D-asset tools in the lecture



I explored two generative AI models. The first one is CLIP (Contrastive Language-Image Pretraining), a text-to-image model that converts raw images into textual descriptions. The second model is Stable Diffusion, which transforms text into images. For intermediate text states, if the user wants to add additional descriptions—such as transforming Charlie Chaplin into a cartoon character and changing the scene to an igloo—these models can achieve style transformation in the resulting images. The effects of the models are demonstrated in the video [HW1–3/2096.MP4](#)